Rotaract Club of Xiamen University Malaysia

Rotary Club of Kuala Lumpur Dynamics

Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

1. TEAM AND PROJECT OVERVIEW

Team Name

T010NG

Project / Solution Name

Kotak

Name (Leader)

Hee Zi Jie

Email & Phone (Leader)

heezijie2004@gmail.com, +60125118986

University Affiliation(s)

Asia Pacific University of Technology & Innovation (APU), University of Reading and University of Sheffield

Live Prototype URL

https://kotak-alaskxntxna.replit.app (mirror: https://kotak.tp073654.workers.dev)

Code Repository URL (public / access granted to judges)

https://github.com/TolongLabs/Kotak

2. ARCHITECTURE & TECHNOLOGY STACK

2.1. System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

A user opens Kotak in the browser. The React app loads dated JSON snapshots built from O*NET, the Anthropic Economic Index and DOSM Job Market Insights, and computes every verdict, fit score and roadmap in the browser with fixed rules. Answers stay in the browser's **localStorage**.

The app calls a Hono API on the same origin. The API matches typed job titles and writes cited explanations with Claude Haiku 5.5, then checks each explanation against the rules and falls back to a template if it fails. It sends Kopi's lines to ElevenLabs for speech, and caches both by a hash of the request. Ask Kopi and the MCP endpoint call 6 read-only tools over the same rules; one tool reads anonymous check-up totals from Neon Postgres.

Before the demo, a RunPod serverless GPU ran Qwen3.8-27B to build each job's 6 skills and 18 questions, which ship inside the snapshots.

GitHub Actions builds and deploys the app and API to Cloudflare Workers on every merge to **main**, and refreshes the data every Saturday. Replit publishes the same code from the team's account. Keys live only in GitHub, Cloudflare and Replit secrets.

System block diagram: see Appendix A.

2. ARCHITECTURE & TECHNOLOGY STACK

2.2. Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
TypeScript 7.0	The language of the whole codebase	Apache 2.0
Bun 1.4	Runtime, package manager, test runner and Replit server	MIT
React 19.3	The web app's user interface	MIT
Vite 8.3	Builds and serves the web app	MIT
Tailwind CSS 4.3	Styling and design tokens	MIT
Recharts 3.10	The Future Skills Radar and the roadmap charts	MIT
Lucide React 1.61	Icons	ISC
pixi.js 6.5 and pixi-live2d-display 0.4	Renders Kopi, the Live2D cat	MIT
Hono 4.13	The API server on Cloudflare Workers and Bun	MIT
Zod 4.6	Validates every request, response and data file	MIT
Anthropic TypeScript SDK 1.1	Calls the Claude API	Apache 2.0

2. ARCHITECTURE & TECHNOLOGY STACK

2.2. Languages, Frameworks & Libraries (continued)

Name & Version	Purpose in Project	License
MCP server SDK 2.3	Serves Kotak's tools to AI assistants at **/api/mcp**	Apache 2.0
Neon serverless driver 1.2	Connects the API to Postgres	MIT
Wrangler 4.151	Deploys to Cloudflare Workers	MIT or Apache 2.0
Playwright Core 1.64 and Clearcote 0.60	Drives the browser for the JobStreet research scraper	Apache 2.0, BSD-3
Live2D Cubism Core for Web	Runtime that plays Live2D models, loaded from Live2D's CDN	Live2D Proprietary Software License
Atkinson Hyperlegible and Shantell Sans fonts	Body, mono and Kopi's handwritten type, from Google Fonts	SIL Open Font License 1.1

3. APIs & CLOUD SERVICES

3.1. External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API and Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Claude API, Anthropic	Matches job titles, explains verdicts and runs Ask Kopi	Paid
Text to Speech API, ElevenLabs	Kopi's voice	Paid
RunPod Serverless API, RunPod	The one-time GPU batch that built each job's skills and questions	Sponsor-provided credit
Google Fonts, Google	Serves the app's fonts	Free
Live2D Cubism CDN, Live2D	Serves the Cubism Core runtime for Kopi	Free

The data sources in section 4.2 are downloaded files, not live APIs: the app never calls them at runtime. API keys are not shown in this document.

3. APIs & CLOUD SERVICES

3.2. Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit Deployments, Replit	Publishes the live demo from **main** on the team's own account	Replit Core (sponsor-provided)
Cloudflare Workers, Cloudflare	The mirror deployment: app, API, KV cache and a Durable Object for MCP and chat	Workers Free; global, Durable Object in APAC
Neon Postgres, Neon	Anonymous check-up totals, research job listings and admin accounts	Free; **aws-ap-southeast-1**, Postgres 18
RunPod Serverless, RunPod	The vLLM GPU endpoint for the one-time batch; 0 workers when idle	Sponsor credit; one H100 80 GB GPU
GitHub and GitHub Actions, GitHub	Code hosting, CI, deploys on every merge to **main** and the Saturday data refresh	Free

4. AI TOOLS, MODELS & DATASETS

4.1. AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Claude Haiku 5.5 (**claude-haiku-5-5**)	Anthropic, Claude API	Pre-trained	Anthropic Commercial Terms of Service
ElevenLabs **eleven_v4** and **eleven_v4_turbo**, voice Johari	ElevenLabs	Pre-trained	ElevenLabs Terms of Service
Qwen3.8-27B-FP8	Qwen, run on RunPod with vLLM	Pre-trained	Apache 2.0

No model is fine-tuned or self-trained.

4. AI TOOLS, MODELS & DATASETS

4.2. Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
O*NET 30.1 Database	https://www.onetcenter.org/database.html	Public	CC BY 4.0	N
Anthropic Economic Index, April and May 2026	https://huggingface.co/datasets/Anthropic/EconomicIndex	Public	Data CC BY; code MIT	N
DOSM Job Market Insights, Q3 2019 to Q2 2026	https://www.dosm.gov.my	Public	DOSM terms, with source credit	N
OpenDOSM skill-related underemployment by age	https://open.dosm.gov.my/data-catalogue/lfs_qtr_sru_age	Public	CC BY 4.0; one figure cited	N
UN SDG indicator 4.4.1, Malaysia	https://unstats.un.org/sdgapi/	Public	UN terms of use; one baseline cited	N
Kotak skills and questions for 923 jobs	Built from O*NET by the RunPod batch	Synthetic, from public data	The team's own work	N
JobStreet job listings	Collected by the team's research scraper from https://my.jobstreet.com	Self-collected	JobStreet's terms apply	N; job ads only

4. AI TOOLS, MODELS & DATASETS

4.2. Datasets (continued)

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Demo check-up profiles	Written by the team	Synthetic	The team's own work	N

MyCOL and Epoch AI are cited as context only; Kotak loads no data from them. The research listings feed no verdict, score or roadmap.

4. AI TOOLS, MODELS & DATASETS

4.3. AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	Used For	Extent of Use (minor / moderate / major)
Claude Code, with Claude models	Code, reviews, research, documentation and pull requests	Major
Codex CLI, OpenCode and Antigravity CLI, with GPT, DeepSeek, GLM and Gemini models	Parallel code changes from written briefs	Moderate
Claude, GPT and Gemini chat models	The field simulation and design review in **docs/research**	Minor

4. AI TOOLS, MODELS & DATASETS

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Every change went through a pull request that ran type-checks and a build in CI before a member merged it. The team checked each feature by running the app in a browser at phone and desktop widths, not by trusting the tool's report. Facts in the app and in these documents link a dated public source. AI-drafted skills carry a visible label in the app until a person reviews them. The team owns every merged line and corrected tool mistakes before merging.

5. SPONSOR-PROVIDED RESOURCES

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	Resource (API / credits / platform / data)	How It Was Used in the Project
Replit	Platform, Core	Hosts the live demo, published from **main** on the team's own account
RunPod	Credits	A serverless H100 ran the batch: 923 jobs, 783 seconds for the full run, about US$1.25
SDC mentors (KrackedDevs)	Mentoring	One mentor session on 23 October 2026; feedback recorded in #98

5. SPONSOR-PROVIDED RESOURCES

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

RunPod was essential to build Kotak's data: it turned O*NET's tasks into 6 skills and 18 questions for each of 923 jobs, which no person could write by the deadline. The batch runs once, so the live app does not depend on RunPod, and the Claude Message Batches API could replace it. Replit hosts the submitted live demo; the same build also runs on Cloudflare Workers, so the hosting is replaceable.

6. SECURITY, PRIVACY & RESPONSIBLE USE

6.1. Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

Kotak has no user accounts. A check-up runs in guest mode: the job, goal and answers stay in the browser's **localStorage**, and Settings > Reset All clears them.

The app sends a typed job title to the Claude API for matching, and a role's skill data for explanations. Ask Kopi sends the user's message, and the results of tools run on their check-up, to Claude. Kopi's lines go to ElevenLabs for speech. Neither request carries a name, email or ID. The API caches answers and audio by a hash of the request.

Postgres stores anonymous check-ups as a role code, 6 skill levels, a sample flag and a date, and shows totals only for groups of 10 or more. The research scraper stores public job ads, not personal profiles. Keys live only in host secrets.

6. SECURITY, PRIVACY & RESPONSIBLE USE

6.2. Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

- **Hallucination**: Claude only explains verdicts that fixed rules set. A check rejects any explanation that changes a verdict or lacks a dated citation, and a template replaces it.
- **Wrong job match**: the user confirms every AI title match.
- **Unreviewed skills**: an open model drafted each job's skills. A validator rejects tasks from other jobs, and the app labels the skills AI-drafted.
- **Thin trend data**: the Economic Index has two monthly points, so 12-month projections are rough. Every target is labelled an estimate, and thin data shows Unknown.
- **Data bias**: O*NET describes US jobs, and DOSM counts only online job ads.
- **Single providers and rate limits**: cached answers, templates and text cover Claude or ElevenLabs outages and HTTP 429 or 529 errors. A limit of 30 Claude calls a minute protects the key.

7. COMPLIANCE & READINESS

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. DECLARATION

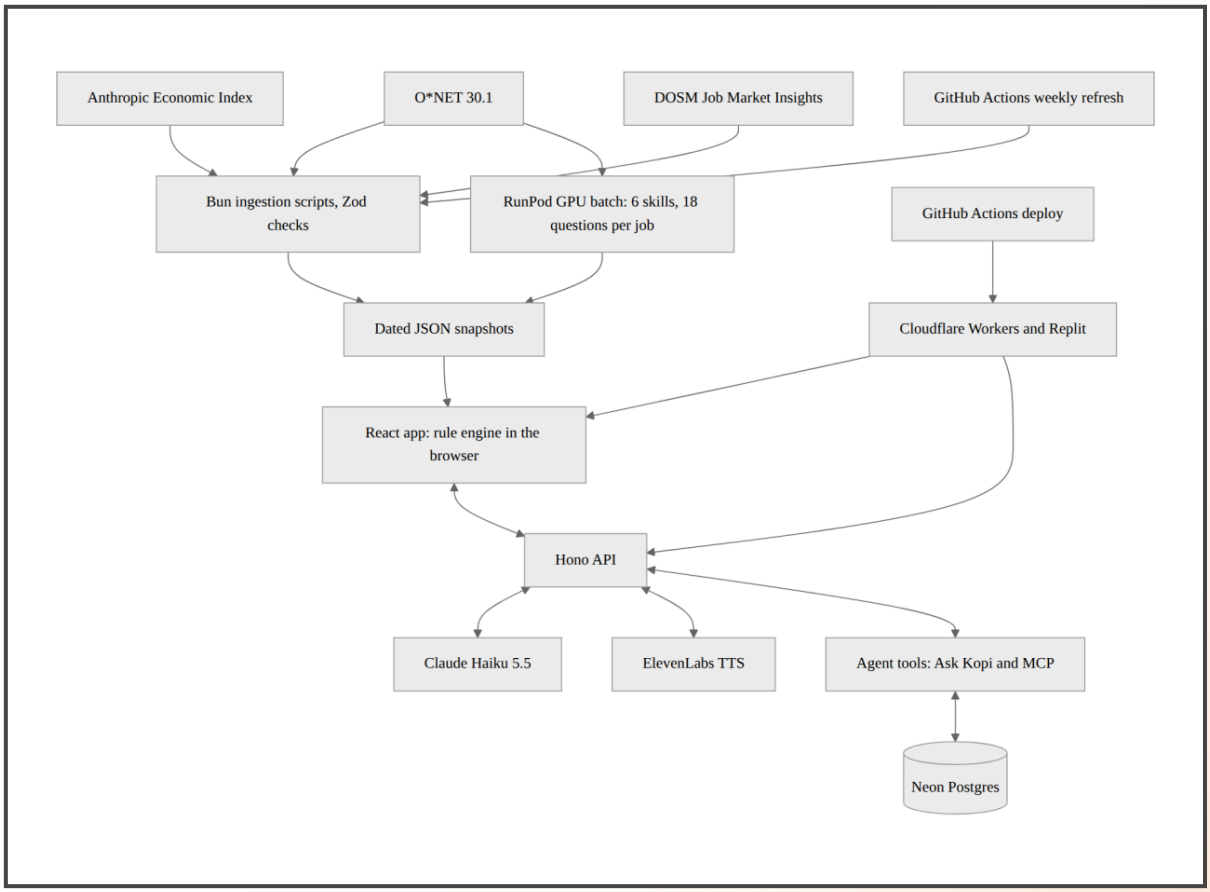
By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



24 October 2026

2. ARCHITECTURE & TECHNOLOGY STACK

A. Appendix A: System Block Diagram



Data flows top to bottom: dated public data and the RunPod batch build the snapshots, the browser computes every verdict, and the API adds Claude, ElevenLabs and the agent tools.

5. SPONSOR-PROVIDED RESOURCES

B. Appendix B: Replit Proof of Use

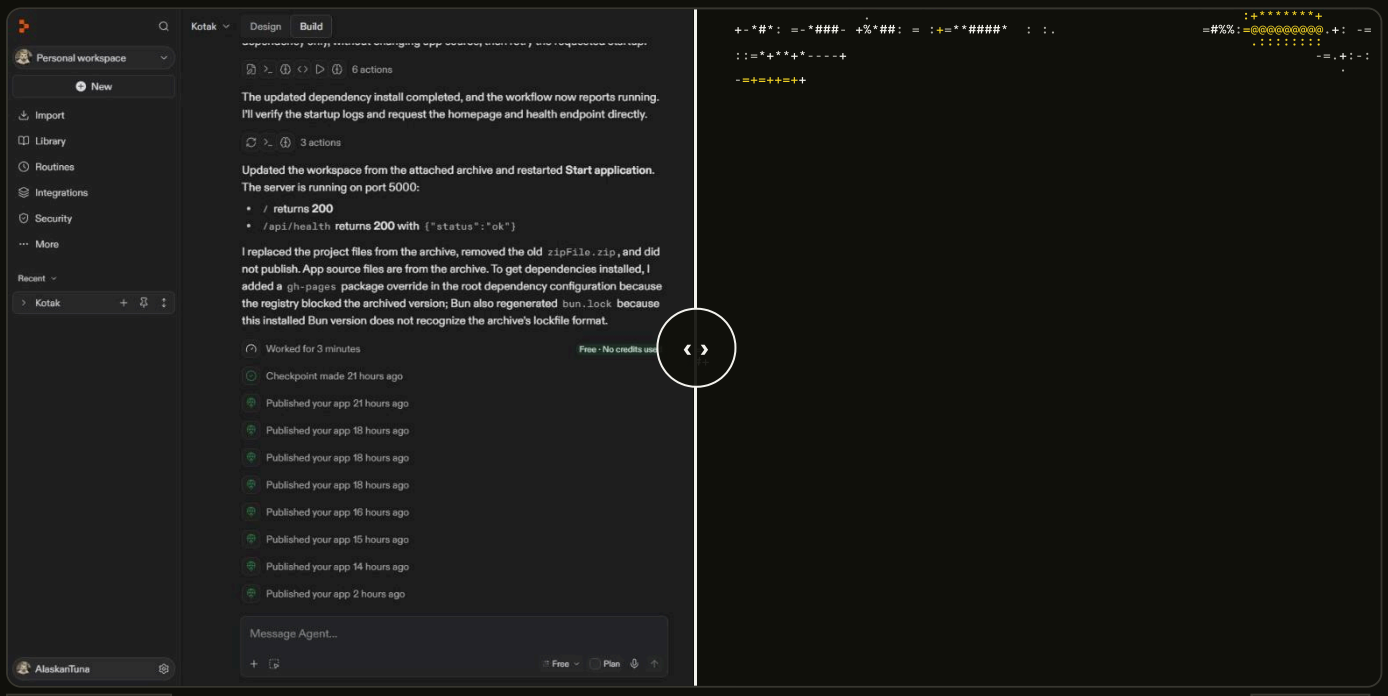
Handbook 10.6 asks for proof that Replit was used, showing the team's own account and project. The following pages show the live deployment at https://kotak-alaskxntxna.replit.app and the project https://replit.com/@alaskxntxna/Kotak with its publish and Git history, from the team leader's account. The same file is **submission/REPLIT PROOF OF USE.pdf** in the repository.

Proof of Use 23 Oct 2026 Status **Live**, public

Replit

Kotak is **live on Replit Autoscale**, published from the team leader's own account and built from the team's GitHub repository.

- kotak-alaskxntxna.replit.app
- @alaskxntxna/Kotak
- Autoscale · Asia



WORKSPACE Fig 1, the Kotak project — the same screen, redrawn in glyphs GLYPHS

01 How Kotak uses Replit p. 2
02 Handbook checklist p. 2
03 How it runs p. 2
04 Replit figures 1–7 p. 3
05 Sources p. 6

Live
public deployment, Replit Autoscale

200
from /api/health, checked 23 Oct 2026

2 vCPU
4 GiB RAM, region Asia

3
secrets in Replit, values hidden

01

How Kotak uses Replit

Source: the Replit workspace, 23 Oct 2026

01	**Replit account**	alaskxntxna (display name AlaskanTuna), the team leader's own account
02	**Project**	https://replit.com/@alaskxntxna/Kotak
03	**Live deployment**	https://kotak-alaskxntxna.replit.app, status Live, sharing Public
04	**Deployment type**	Replit Autoscale, region Asia, 2 vCPU / 4 GiB RAM
05	**Source**	GitHub repository TolongLabs/Kotak (named sdc-hack when captured), connected through Replit Git (branch `github-main`)
06	**Run configuration**	`.replit`: `bun install`, `bun run build`, then `bun run start` on port 5000
07	**Secrets**	`ANTHROPIC_API_KEY`, `ELEVENLABS_API_KEY` and `SESSION_SECRET` in Replit Secrets; values stay hidden
08	**Publish history**	9 publishes by AlaskanTuna between 22 and 23 October 2026

02

Handbook checklist

Handbook 10.6 asks for any one proof; this document gives two

- [x] **A live deployment Replit URL** — Fig 2 1 2 3 · Fig 7
  `kotak-alaskxntxna.replit.app` is Live and Public, and serves Kotak.
- [x] **A Replit project link with its history** — Fig 1 2 4 · Fig 3 1 · Fig 5 2
  `replit.com/@alaskxntxna/Kotak`, with its publish and Git history.
- [x] **The team's own account** — Fig 1 1 · Fig 2 1 · Fig 4 3
  AlaskanTuna, the team leader, is signed in and publishes the app; commits go through the linked GitHub profile.
- [x] **The team's own project** — Fig 4 1 2
  The project's Git remote is the team's repository, TolongLabs/Kotak (then sdc-hack), through an active GitHub connection.

03

How it runs

Config: .replit in the repository root

.replit — deployment

```
[deployment]
deploymentTarget = "autoscale"
build = ["sh", "-c", "bun install && bun run build"]
run = ["sh", "-c", "PORT=5000 bun run start"]
```

health.sh — checked 23 Oct 2026

```
curl https://kotak-alaskxntxna.replit.app/api/health

# response
{"status":"ok"}
```

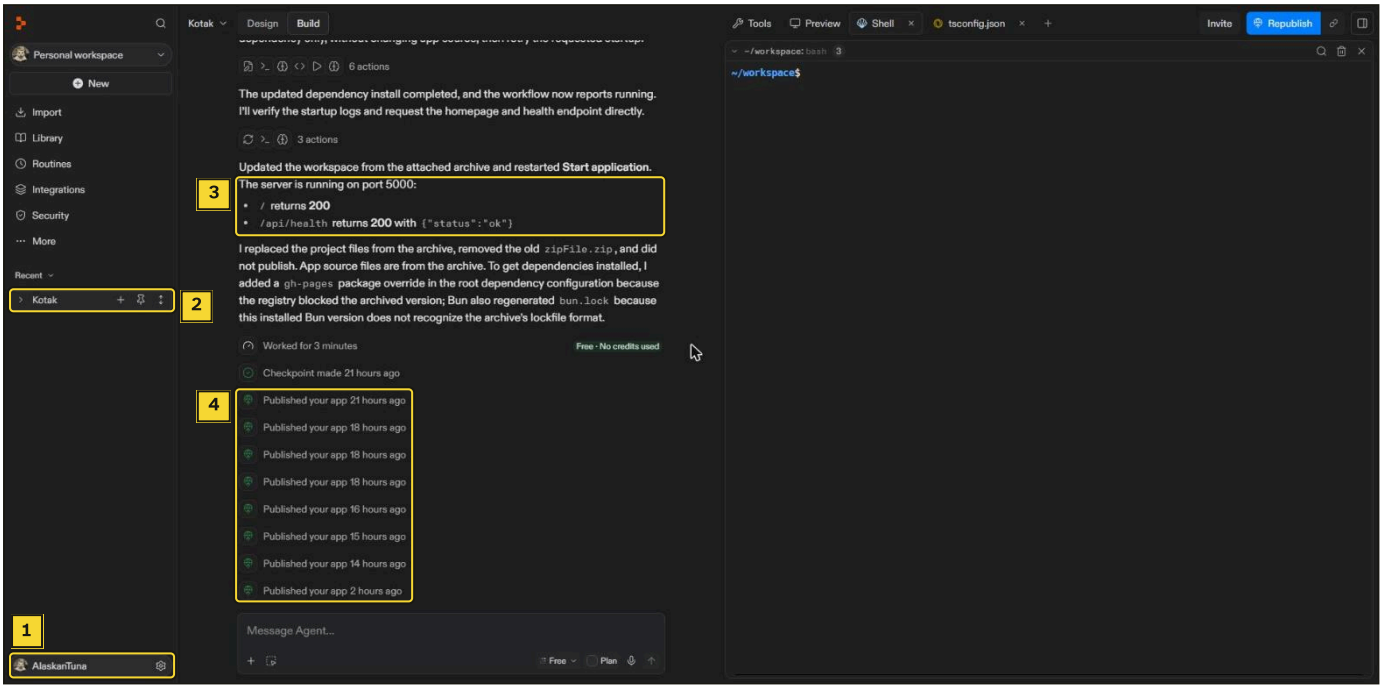
04

Replit figures

Captured from the team leader's account, 23 Oct 2026

FIG 1 The Kotak project

Build view



1 Signed in as AlaskanTuna, the team leader.

2 The Kotak project in the personal workspace.

3 Replit Agent's check: the server runs on port 5000 and /api/health returns 200.

4 Publishes, from 21 hours to 2 hours before capture.

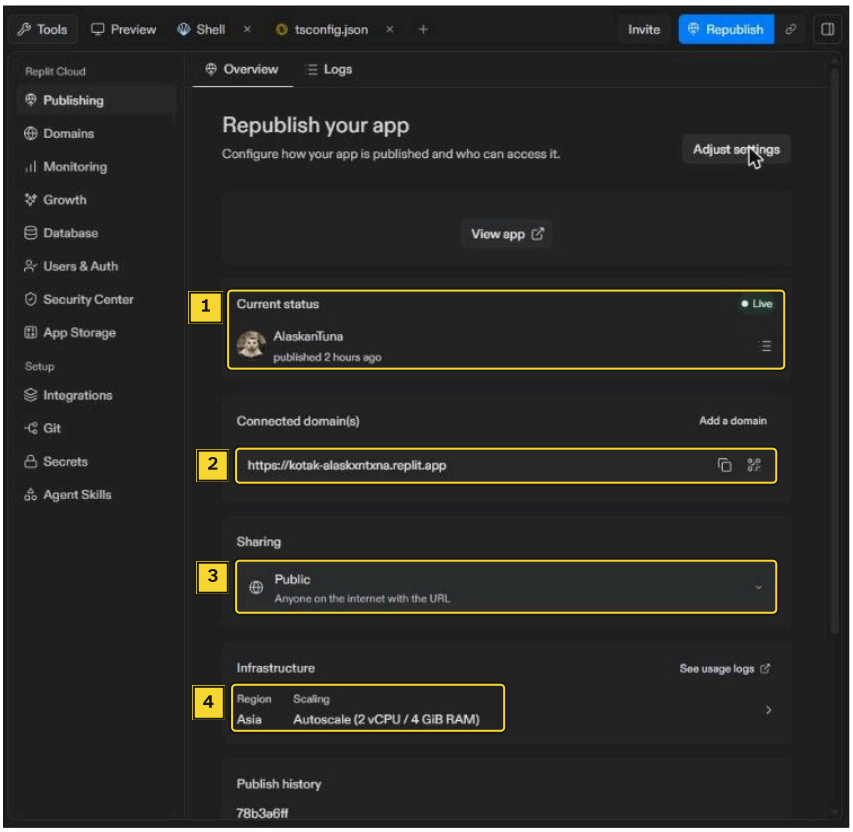


FIG 2

Publishing

Overview tab

1 Current status Live, published by AlaskanTuna.

2 The deployment URL, kotak-alaskxntxna.replit.app.

3 Sharing Public: anyone with the URL can open it.

4 Region Asia, Autoscale with 2 vCPU and 4 GiB RAM.

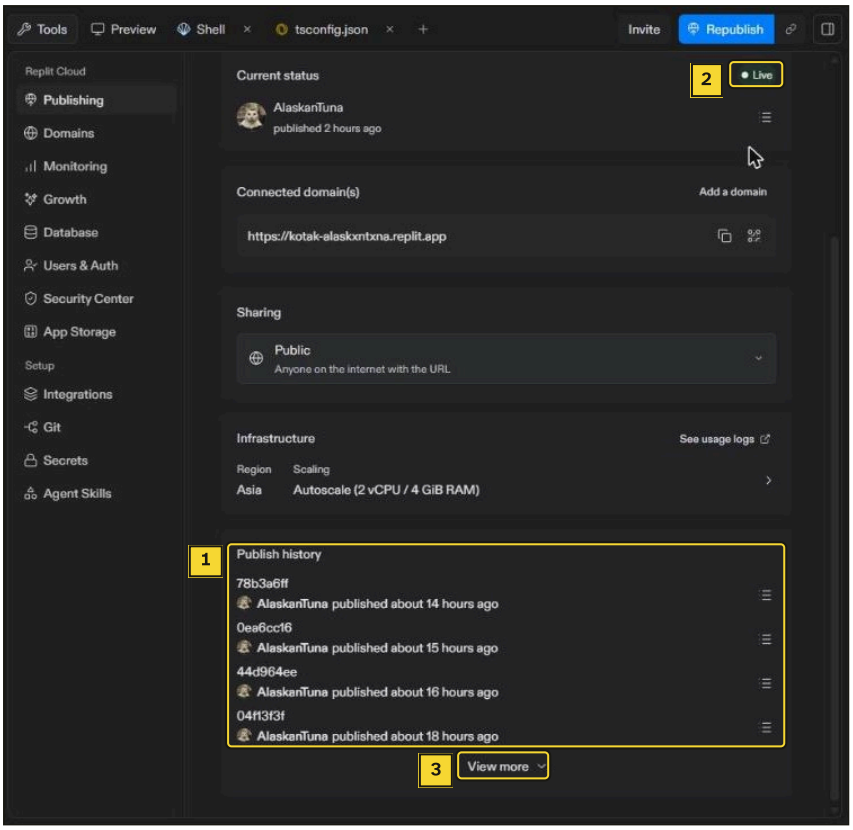


FIG 3

Publish history

`Publishing, scrolled down`

1. Each publish carries its commit (78b3a6ff, 0ea6cc16, 44d964ee, 04f13f3f) and AlaskanTuna.
2. The deployment is still Live.
3. Older publishes sit behind View more.

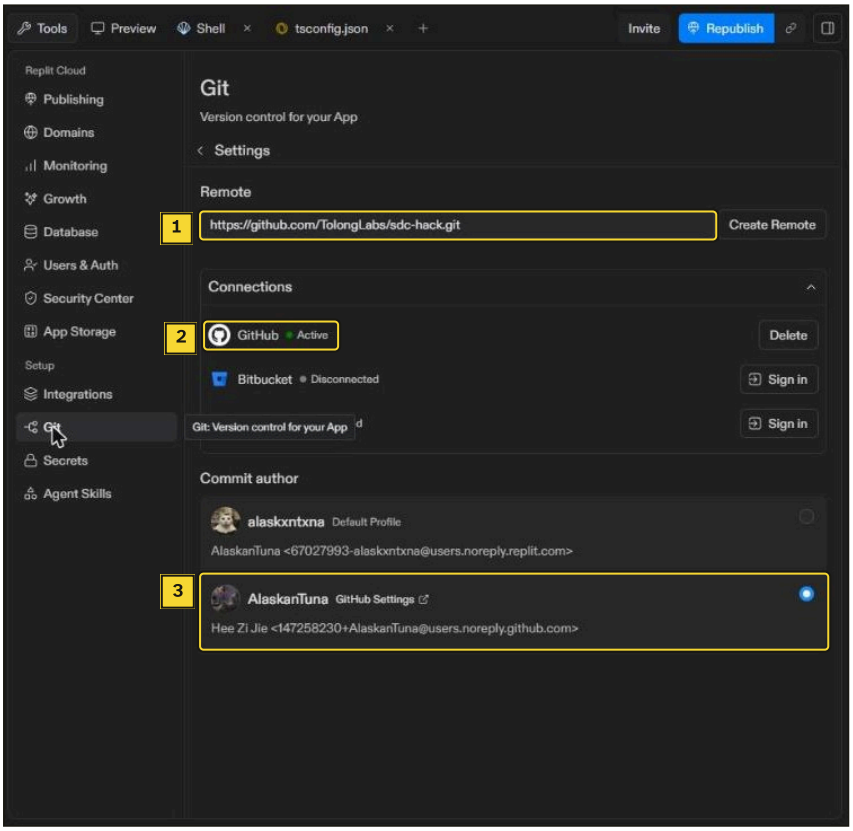


FIG 4

Git settings

`Git, Settings`

1. The remote is the team's repository, github.com/TolongLabs/sdc-hack.git.
2. The GitHub connection is Active.
3. The selected commit author is the linked GitHub profile, AlaskanTuna.

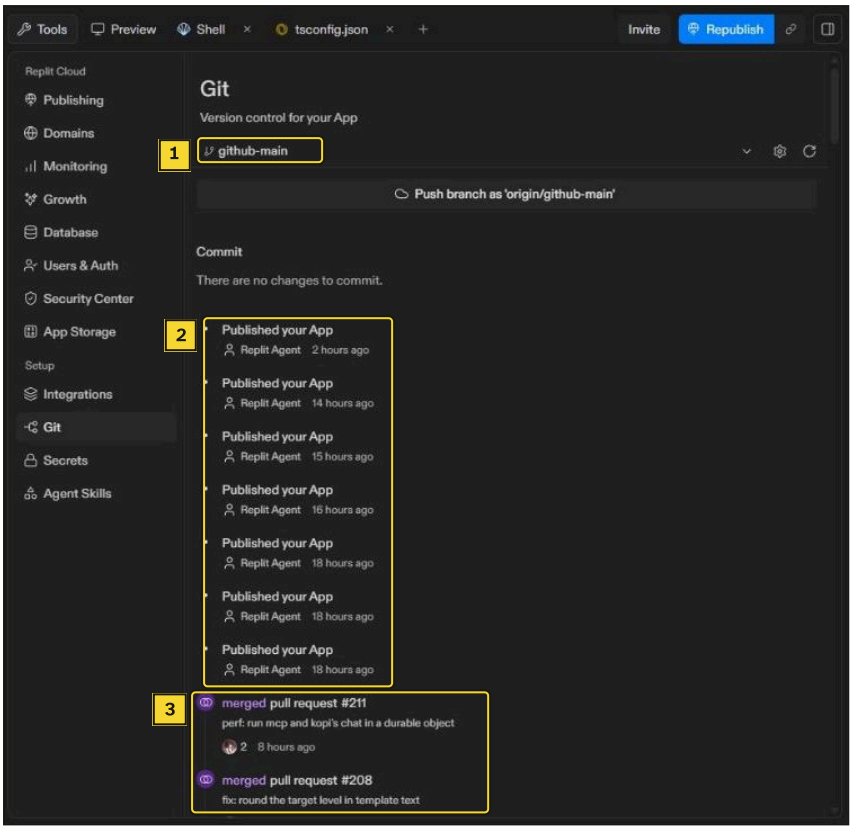


FIG 5

Git history

`branch github-main`

1. Branch github-main, tracking the team's repository.
2. Publishes by Replit Agent, the newest 2 hours before capture.
3. The merged pull requests they include, such as #211 and #208.

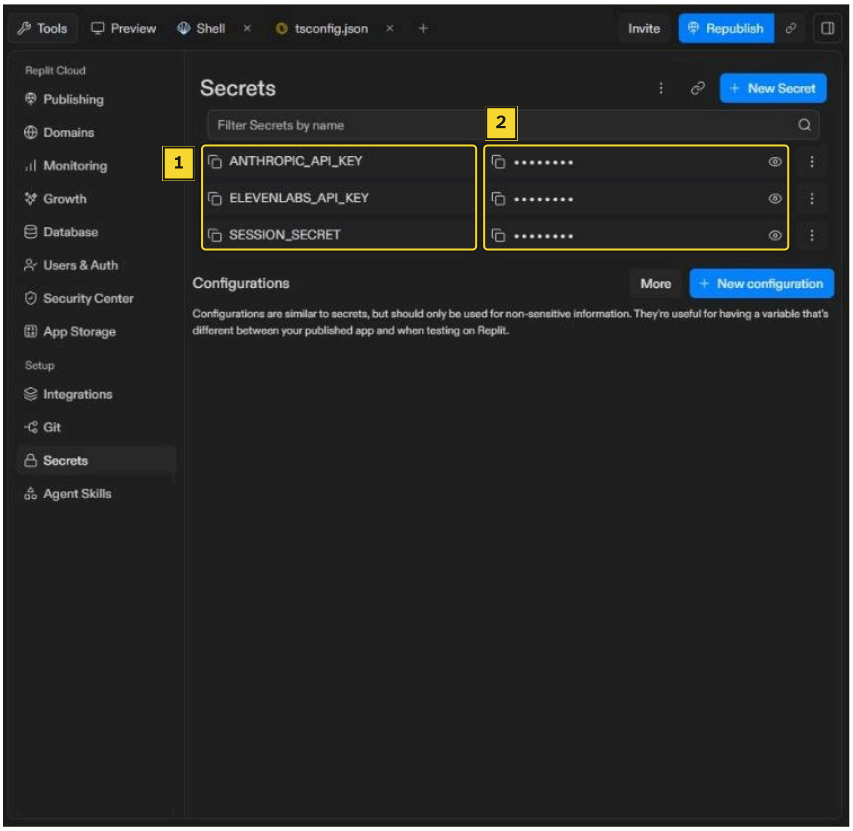


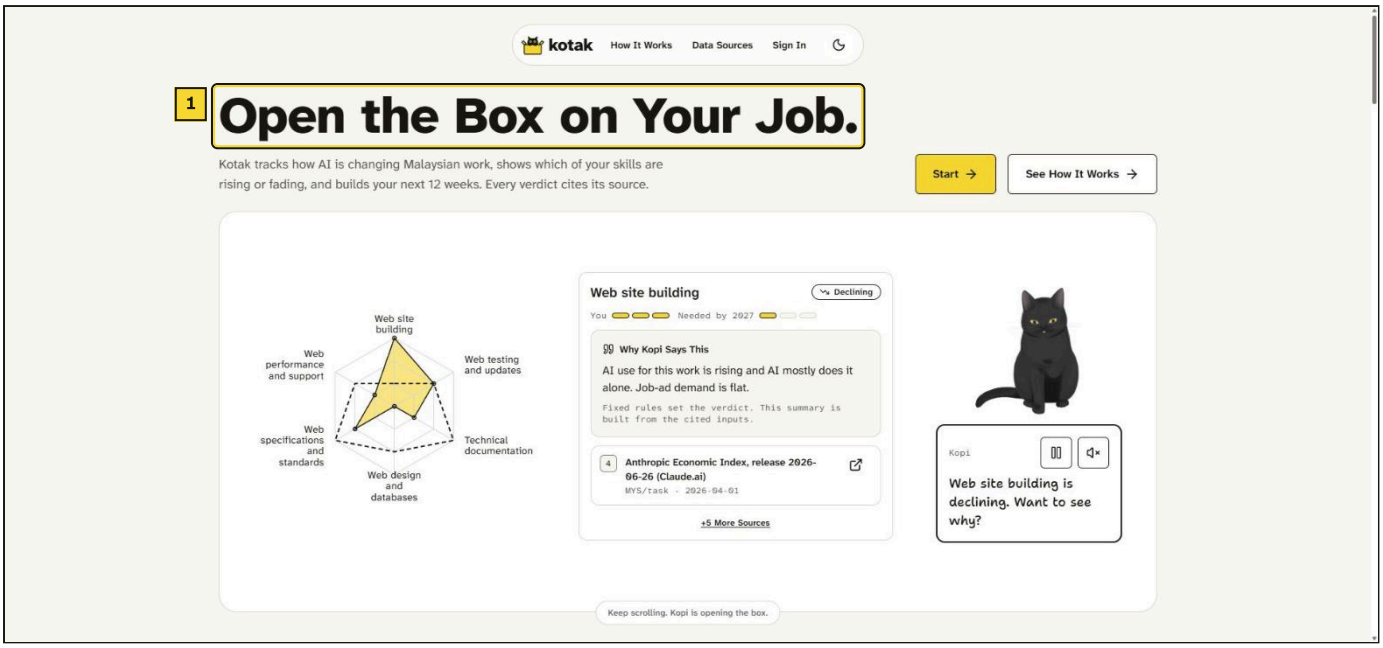
FIG 6

Secrets

`Secrets tab`

1. ANTHROPIC_API_KEY, ELEVENLABS_API_KEY and SESSION_SECRET.
2. Values stay hidden, and none are in the code.

FIG 7 **The published app** 23 Oct 2026



1 Kotak's landing page, opened from the deployment in Fig 2 on 23 Oct 2026.

05 · Sources

Open it yourself: Kotak is live on Replit.

Live app	kotak-alaskxntxna.replit.app
Project	replit.com/@alaskxntxna/Kotak
This PDF	scripts/proof-of-use/replit.html
Repository	github.com/TolongLabs/Kotak



5. SPONSOR-PROVIDED RESOURCES

C. Appendix C: RunPod Proof of Use

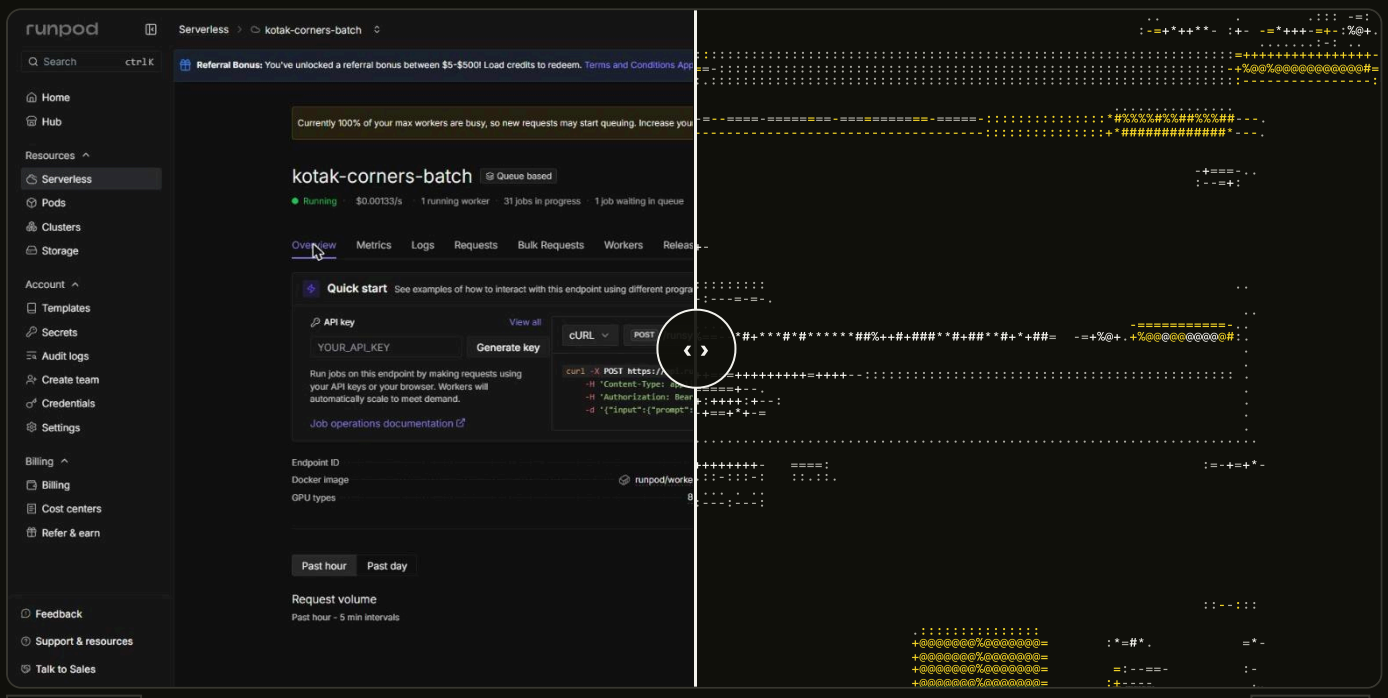
Handbook 10.7 asks for a RunPod dashboard screenshot showing the endpoint URL, the GPU type and the runtime, from the team's own account. The following pages show the serverless endpoint kotak-corners-batch (**https://api.runpod.ai/v2/5yjk19uqmrpbp7**) on one NVIDIA H100 80GB GPU, and the batch runtime. The same file is **submission/RUNPOD PROOF OF USE.pdf** in the repository.

Proof of Use 22 Oct 2026 HTTP errors 0 of 930

RunPod

One serverless H100 run drafted the skill corners for **all 923 O*NET occupations** Kotak checks people against, ahead of time and once.

- kotak-corners-batch
- Qwen3.8-27B-FP8
- 1 × H100 SXM 80 GB



CONSOLE Fig 1, the endpoint overview — the same screen, redrawn in glyphs GLYPHS

01 What ran on RunPod p. 2
02 Handbook checklist p. 2
03 Workload p. 3
04 Example request p. 3
05 Console figures 1–5 p. 4
06 Sources p. 6

923	1,285 s	2.23 M	0
occupations drafted	GPU runtime, smoke + full run	tokens in and out of Qwen3.8-27B	roles failed; 7 answers redrawn

01 What ran on RunPod

Source: RunPod console and `data/generated/role-corners-run.json`

01	**RunPod account**	Team account (AlaskanTuna), funded by the SDC RunPod voucher: US$25 credit redeemed 22 Oct 2026
02	**Product**	Serverless, queue-based endpoint `kotak-corners-batch`
03	**Endpoint ID**	`5yjk19uqmrpbp7`
04	**Endpoint URL**	`https://api.runpod.ai/v2/5yjk19uqmrpbp7` (OpenAI-compatible route `/openai/v1`)
05	**Worker image**	`runpod/worker-v1-vllm:v2.28.0` (vLLM v0.30.0, CUDA 13.0), the stock RunPod vLLM worker
06	**Model**	`Qwen/Qwen3.8-27B-FP8`, open weights under Apache 2.0
07	**GPU**	1 × NVIDIA H100 80GB HBM3 (H100 SXM), data centre AP-IN-1; pools 80 GB PRO and 80 GB; max 1 worker
08	**Runtime**	1,285 s in total: smoke run 502 s (with cold start), full run 783 s
09	**Volume**	930 requests, 0 HTTP errors, 7 answers rejected by the validator and regenerated, 0 roles failed
10	**Tokens**	1,092,522 prompt, 1,137,450 completion
11	**Cost**	About US$1.25: account balance US$25.00 before, US$23.75 after (RunPod console, Billing page, 22 Oct 2026 16:05 MYT)

02 Handbook checklist

Participant Handbook sections 10.7 and 9.4

[x] Endpoint URL, in place of a Pod ID — Fig 1 [1] [2]

10.7: a Serverless team shows its endpoint URL. The endpoint ID and the `api.runpod.ai/v2/5yjk19uqmrpbp7` URL both appear on the endpoint page.

[x] GPU type — Fig 1 [1] · Fig 2 [2]

The endpoint's GPU pools (80 GB Pro, 80 GB), and the H100 SXM the worker actually ran on.

[x] Runtime — Fig 2 [3] · Fig 4 [2]

The worker's uptime mid-run (11 m 16 s) and the endpoint's execution time. The run record gives the exact total: 1,285 s.

[x] The team's own account — Fig 1 [5] · all figures

Every figure is the same signed-in console, with the account's balance and avatar top right.

[x] GPU work runs on RunPod — Section 03, Workload

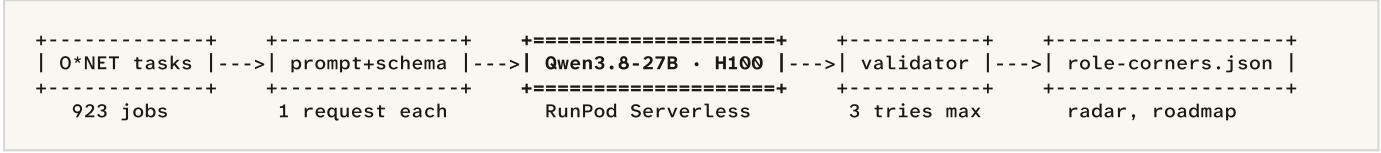
9.4: this batch is Kotak's only GPU work. No model is fine-tuned or self-trained, and the live app calls the Claude and ElevenLabs APIs instead of running a model.

03

Workload

Code: scripts/corners/run-batch.ts
Notes: runpod/README.md

Kotak turns any job into six skill corners and asks twelve yes/no questions about them. RunPod generates that content once, ahead of time, for every one of the 923 O*NET occupations. There is no custom handler: the stock RunPod vLLM worker serves the model. This is Kotak's only GPU work: the live app calls the Claude and ElevenLabs APIs (Technical Declaration, sections 3 and 4).



01	**Input**	The job's title, description and O*NET tasks, grouped by work activity into at most six groups by fixed rules, with no AI.
02	**Ask**	The model groups the tasks into six named corners and writes three can-do statements per corner (basic, working, advanced), with learning hours, SDG targets and search-link resources.
03	**Constrain**	A strict JSON schema goes in `response_format`; vLLM's structured outputs make every token follow it.
04	**Check**	A validator rejects any answer that drops a task, puts a task in two corners or cites a task the job does not have.
05	**Retry**	A rejected answer goes back to the model with its errors. Seven were regenerated; none failed.
06	**Write**	`data/generated/role-corners.json` feeds the check-up questions, the Future Skills Radar and the roadmap; `role-corners-run.json` records every run.

04

Example request

Endpoint settings: runpod/README.md

request.sh — POST /openai/v1/chat/completions

```
curl https://api.runpod.ai/v2/5yjk19uqmrpbp7/openai/v1/chat/completions \
  -H "Authorization: Bearer $RUNPOD_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "Qwen/Qwen3.8-27B-FP8",
    "messages": [{"role": "user", "content": "Name one skill of a graphic designer as JSON."}],
    "temperature": 0,
    "max_tokens": 50,
    "chat_template_kwargs": {"enable_thinking": false},
    "response_format": {"type": "json_schema", "json_schema": {"name": "skill", "schema": {
      "type": "object", "properties": {"skill": {"type": "string"}},
      "required": ["skill"], "additionalProperties": false}}}
  }'
```

rerun.sh — RUNPOD_API_KEY and RUNPOD_ENDPOINT_ID set

```
# smoke run on the three demo roles, then the full run; it skips jobs already done
bun run batch:corners --codes 15-1254.00,15-1255.00,43-3031.00
bun run batch:corners
```

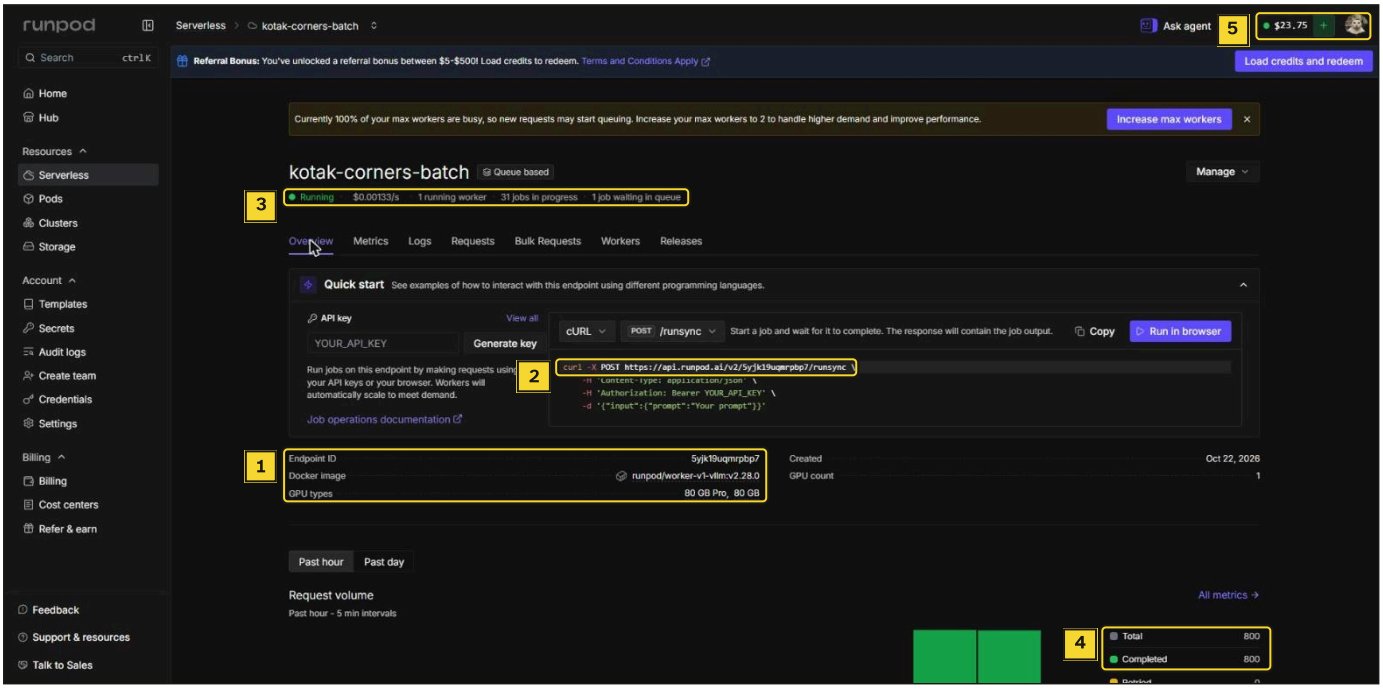
05

Console figures

Captured during the full run,
22 Oct 2026, about 16:00 MYT

FIG 1 Endpoint overview

16:00 MYT



1 Endpoint ID 5yjk19uqmrpbp7, the worker image and the GPU pools: 80 GB Pro, 80 GB.

2 The endpoint URL, in RunPod's own cURL example.

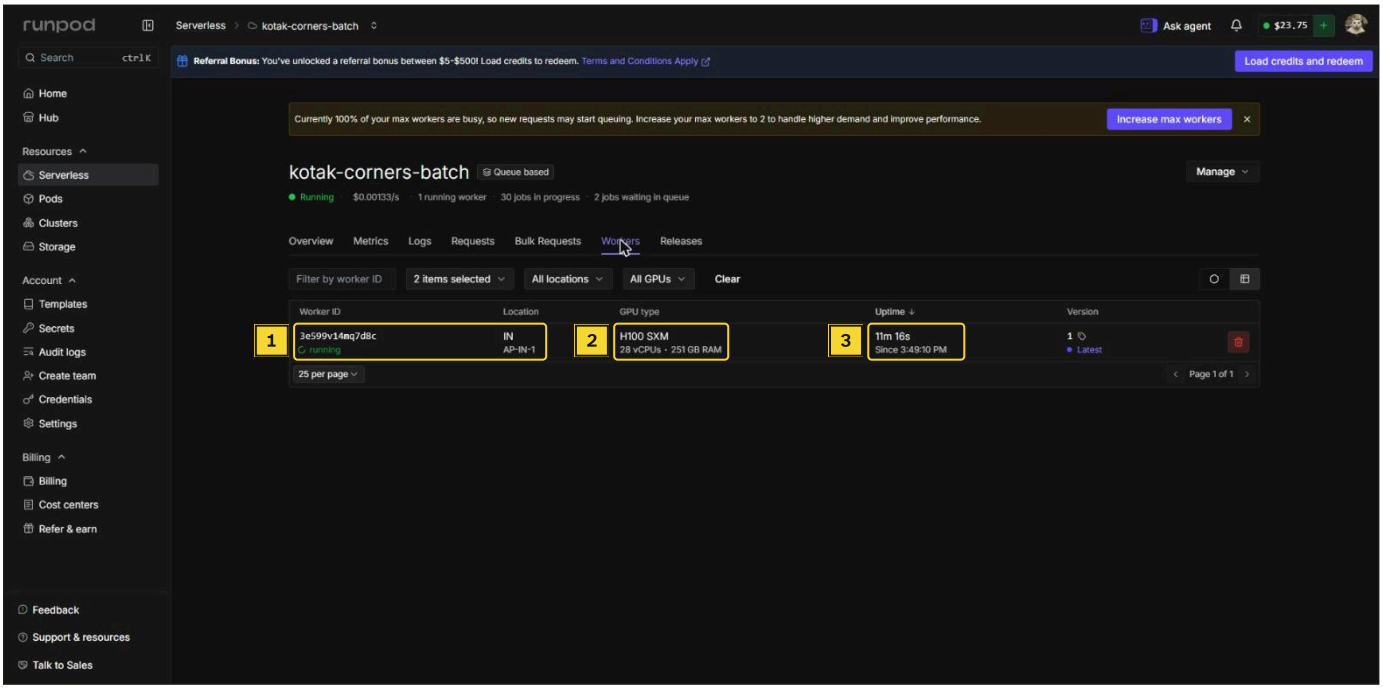
3 Running, with 1 worker and 31 jobs in progress.

4 800 requests completed in the past hour.

5 The account's balance, US$23.75, and avatar.

FIG 2 Workers tab

during the full run



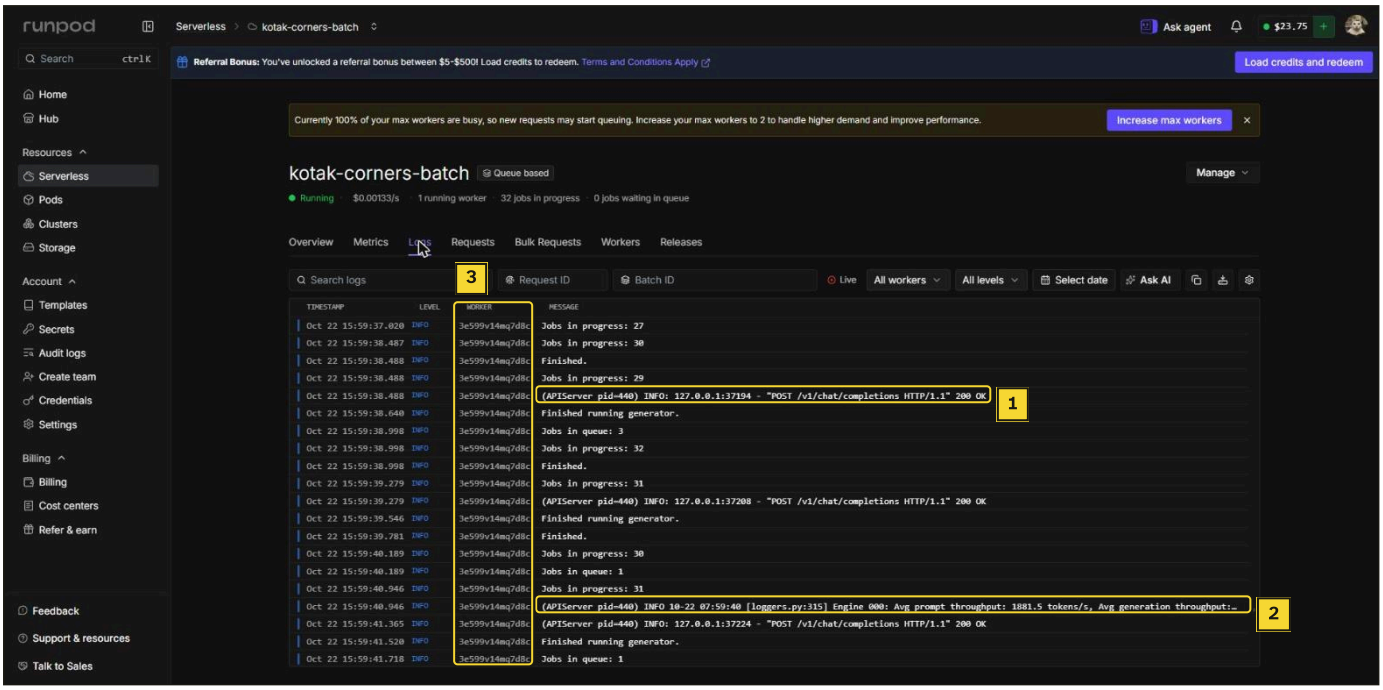
1 Worker 3e599v14mq7d8c, running in AP-IN-1.

2 GPU type: H100 SXM, 28 vCPUs, 251 GB RAM.

3 Uptime 11 m 16 s, since 3:49:10 PM.

FIG 3 Logs tab

15:59 MYT



1 vLLM serving `POST /v1/chat/completions` with 200 OK.

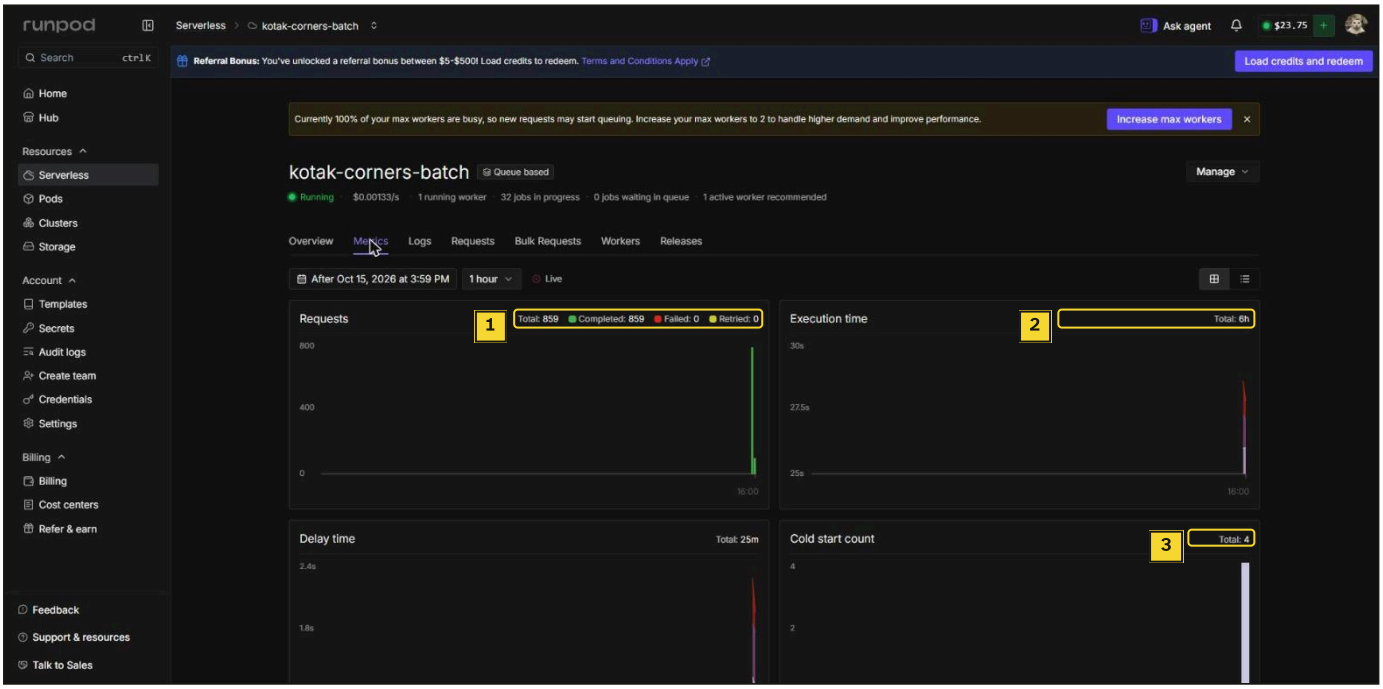
2 The engine's average prompt throughput, 1,881.5 tokens/s.

3 Every line comes from worker 3e599v14mq7d8c.

About 30 jobs are in progress at once: the runner sends 32 requests in parallel.

FIG 4 Metrics tab

at capture time



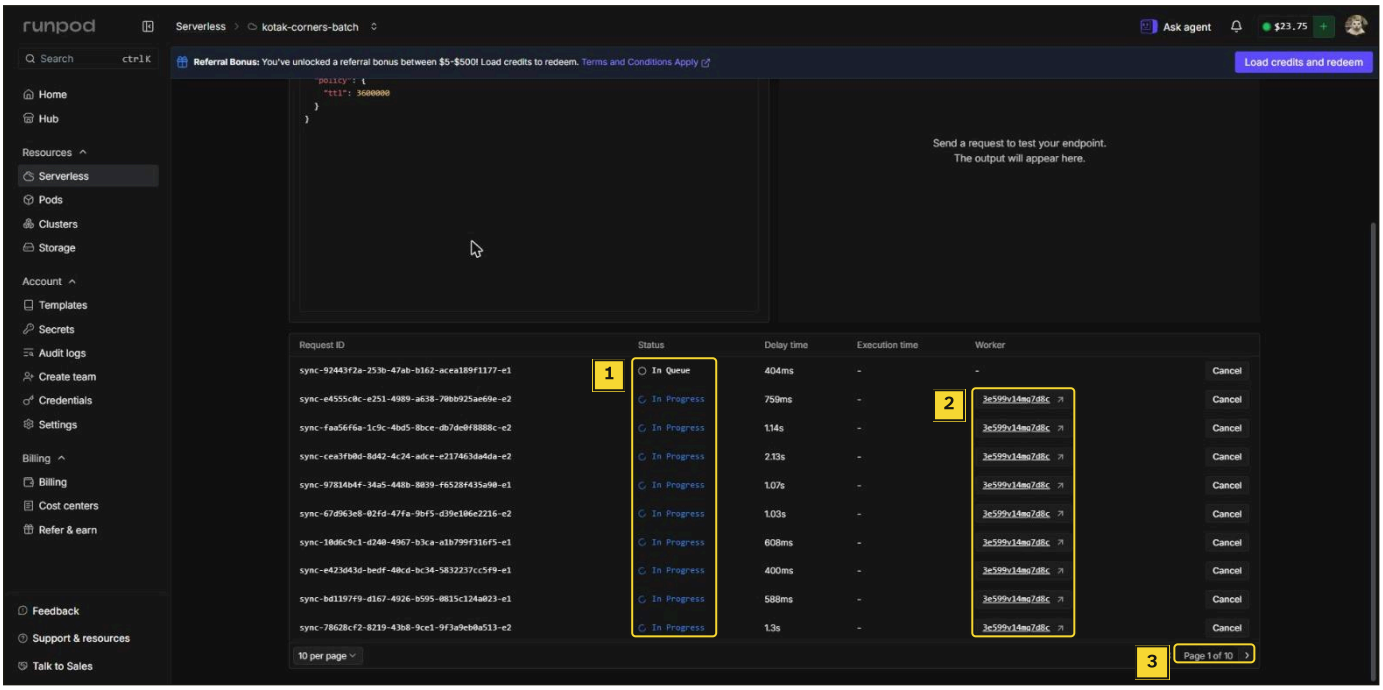
1 859 requests completed, 0 failed, 0 retried.

2 Execution time, 6 h summed over parallel jobs.

3 4 cold starts across both runs.

The filter starts on 15 Oct, so the totals cover both runs; the bars sit at 16:00 on 22 Oct.

FIG 5 **Requests tab** during the full run



1 Batch jobs in progress and in queue.

2 Each one on worker 3e599v14mq7d8c.

3 Page 1 of 10 of the request list.

06 · Sources

Where every number here comes from.

Console	Figures 1–5, 22 Oct 2026
Run record	data/generated/role-corners-run.json
Deploy notes	runpod/README.md
Credit, cost	Handbook 9.1.2; RunPod Billing page
This PDF	scripts/proof-of-use/runpod.html
Repository	github.com/TolongLabs/Kotak

